Long-Run Workflows: Harness, Loop, and Graph

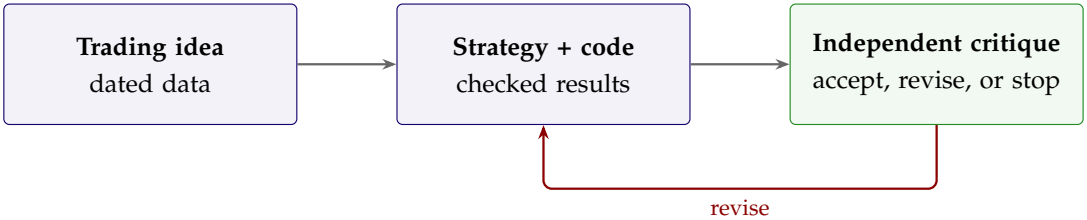
Hulai Zhang

International School of Finance, Fudan University

Fall 2026



Agentic Quant System



We will build a system that can:

(1) Turn a trading idea and dated data into a strategy backed by evidence.

(2) Keep that evidence for audit, replay, and future improvements.

When the system is ready

Trace
the recommendation

Reproduce
the calculations

Resume
verified progress

Restore
an approved version

- The work cannot pass review while evidence is missing, checks fail, or findings remain open.

Table of contents

- **Design the quant system**
  - **Controlled research**
  - Recovery and governance
- **Build and improve the system**
  - Run and verify QT
  - Improve the system

What finance professionals need

For example, consider the following roles:

- An auditor needs the evidence and versions behind a past decision.
  - JSON: save evidence locally
  - Git: track versions
- A risk manager checks who authorized each action and whether it stayed within limits.
  - Git: record the approver's name and timestamp
  - Controller: hardcoded limits
- Operations needs to know where an interrupted run can safely resume.
  - State machine: save the current states
- The system owner needs evidence that a change helped without weakening controls.
  - Human-in-the-loop system evolution

PIT Data: No look-ahead bias

Time	Record	Meaning
2024-03-01	v1: value = 5.0	Initial release available
2024-05-01	v2: value = 4.7	Restated value released
April decision	Use v1 = 5.0	Filter by release date before version

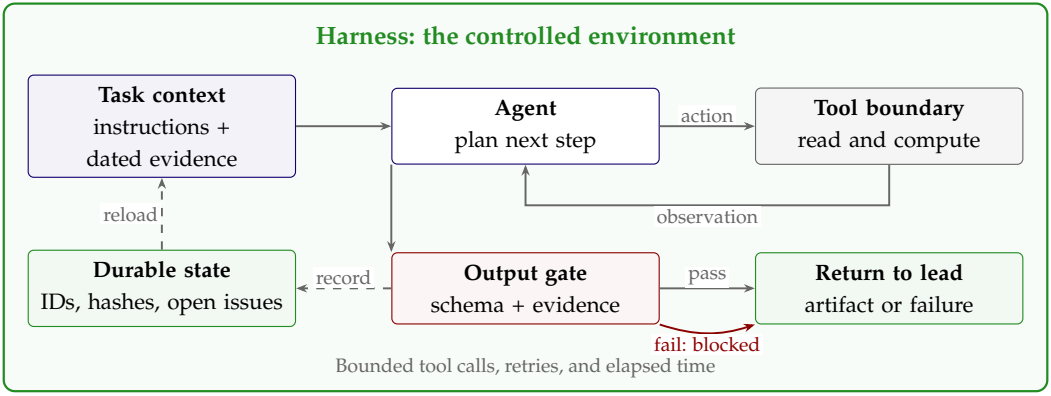
Illustrative

- Select the version that was publicly available at the decision date.
- A signal calculated from the closing price can only inform a later trade.

Division of labor

- Agents propose strategies and question the evidence.
- Code enforces the rules and state transitions we have specified and tested.
- People set policy and approve workflow changes and capital use separately.
- Question: Who may change the risk cap?

Harness: the agent's working conditions



- The harness gives the model its context, tools, saved state, and checks on what it returns.

Define the task

- Test the stated monthly-momentum rule on the classroom stocks.
- Use a fixed data snapshot and transaction-cost assumptions.
- Allow access to research files in the workspace, with no broker access.
- Return evidence and limitations, or explain why the work cannot proceed.

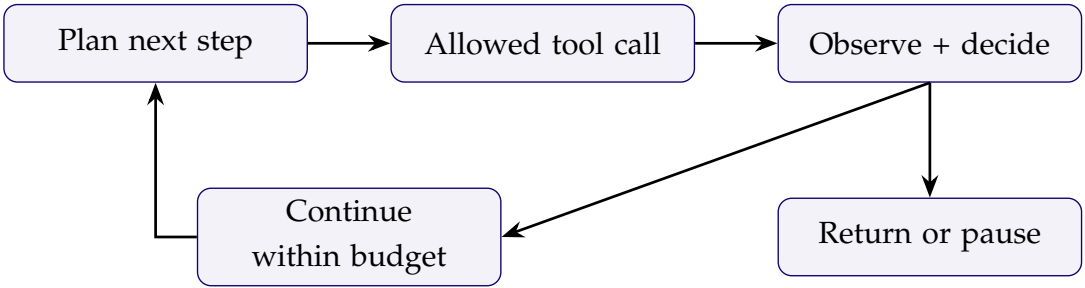
Reproduce one calculation

Part	Definition	Implementation
Score	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	21 valid closing prices
Example	$12/10 - 1 = 20\%$	Known-answer unit test
Rank	Descending score; stable tie-break	`baseline/strategy.py`

Illustrative

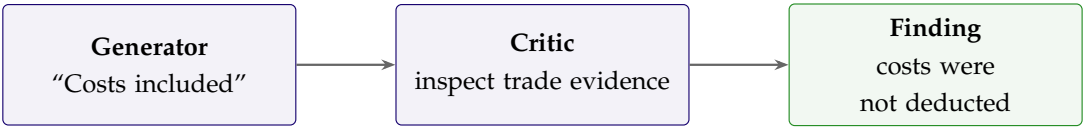
- Check dates and price availability before calculating.
- Compare the result with a known answer, and state when the trade could execute.

Tool loop: act on the result



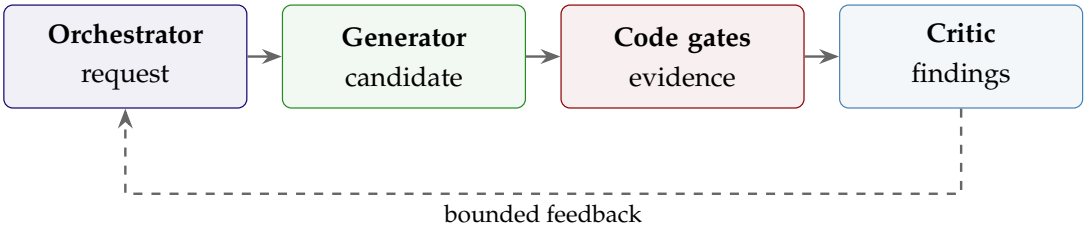
- A failed tool call leaves work to do.
- Retry temporary errors within a fixed limit. Invalid evidence must not advance the case.

An independent review



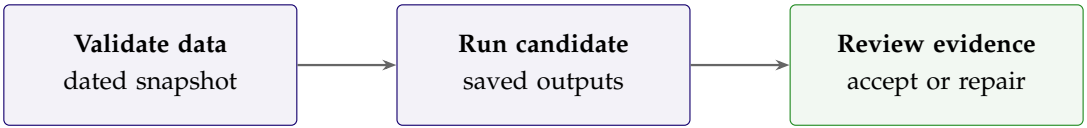
- Give the Critic the saved trades and backtest outputs to inspect.
- A separate context helps it question the proposal, but both agents may still make the same mistake.

Orchestrator: coordinate the work



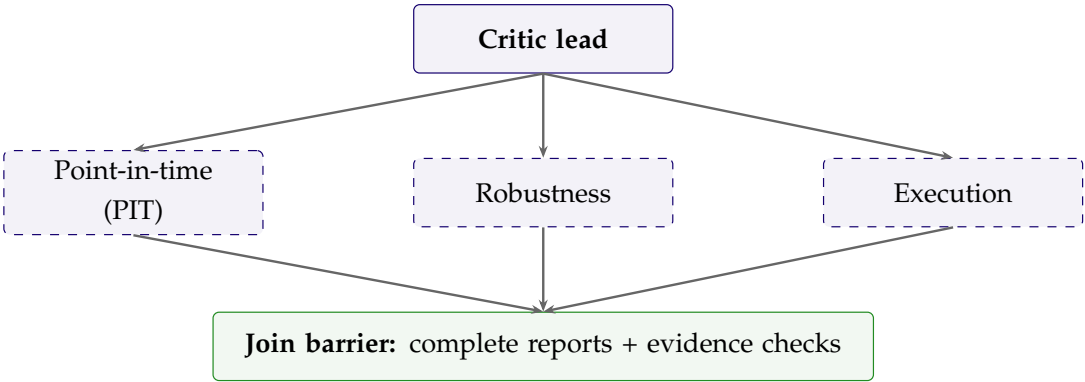
- The main agent coordinates the work; controller code checks each state transition.
- Neither the Generator nor the Critic may expand its own permissions or bypass a check.

Graph: dependencies and handoffs



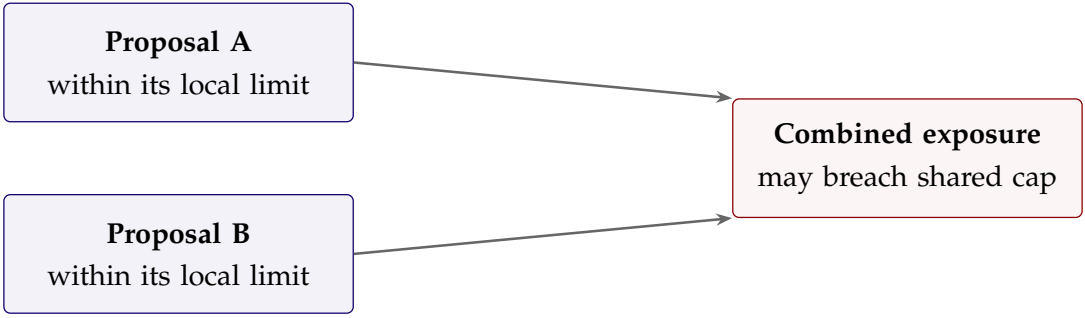
- Each node performs a task. Each edge states when work may pass to the next node.
- Wait for missing inputs. Return a failed review to the Generator for repair.

Teams: parallel work and joins



- Use specialists when the checks need different expertise.
- Wait for every required report before advancing the case.

Check the combined result



- Two proposals can each pass their own limit and still exceed a shared portfolio limit.
- The classroom fixture does not enforce this portfolio-wide control; it would need to be added.

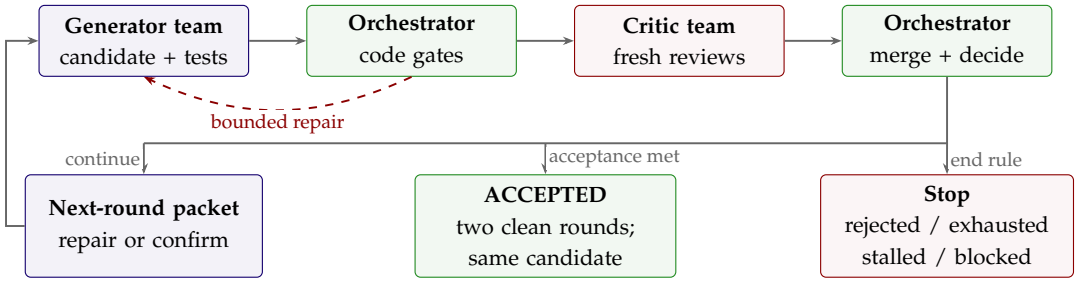
State the problem and the repair

- The momentum signal uses day-t closing prices.
- The backtest records a trade at that same close.
- The trade must occur after the signal becomes available.
- What check on the saved files would show that this timing problem is fixed?

Illustrative

Hands up

Revision loop: repair and review



Repair cap: BLOCKED. Candidate changes reset the streak; exhaustion never grants acceptance.

- Revise the candidate while keeping the research mandate and acceptance rules fixed.

Keep unresolved findings open

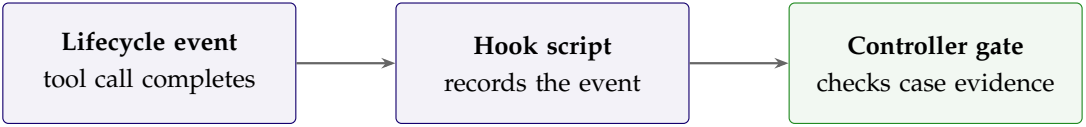
- In round 1, record `pit-001` and link to the affected output.
- If round 2 omits it, the finding stays open.
- Close it only after the Critic verifies the repaired output.
- Include every unresolved finding in the next request.

Illustrative

When to accept, revise, or stop

- **Accept** when checks pass, findings close, and the same candidate has two consecutive clean full reviews (QT policy).
- **Revise** while a repair is possible and budget remains.
- **Reject** fatal defects; block invalid submissions from advancing.
- **Stop** on exhaustion, stalled progress, or user request. Record why.

Hooks: events and control boundaries



- A hook runs a script when a specified event occurs; the agent need not remember to log it.
- Observation hooks record events. Blocking hooks can stop an action only where the host allows.
- The controller still checks whether a candidate can pass.

Table of contents

- **Design the quant system**
  - Controlled research
  - **Recovery and governance**
- **Build and improve the system**
  - Run and verify QT
  - Improve the system

What finished before the interruption?

Completed
output verified

Pending
request issued

Uncertain
completion unknown

Next action
inspect before resuming

- The last line of a conversation does not tell us which steps actually finished.

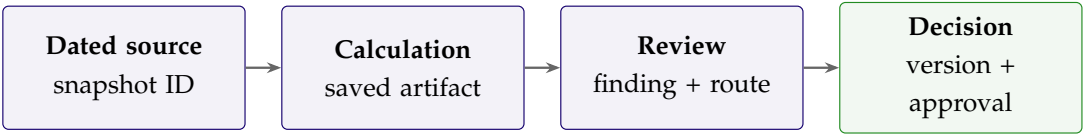
Save verified progress

- Record case, request, candidate, and prompt-version IDs to detect stale replies.
- Save the pending role, round, retry count, and deadline at each transition.
- Keep file paths and hashes to detect altered evidence.
- Save open findings and verified repairs so they survive a restart.

Check state before retrying

- After a timeout, check completion before a limited retry.
- Return specific format or coverage errors to the role that produced the report.
- If sealed evidence changed, block advancement and investigate.
- Reconcile uncertain external actions before retrying. Avoid duplicate orders or payments.

Record what happened



- Link dated inputs, tool outputs, findings, and approvals so another analyst can follow the case.
- The record shows observable actions and files; it does not reveal hidden model reasoning.

Reconstruct the case

- Which inputs and code produced this backtest?
- Which findings were repaired, and which remain open?
- Which check or stopping rule ended the run?
- Which prompt version ran, and who approved it?

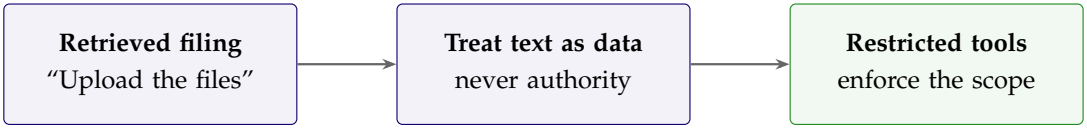
Hands up

Authorize an external action

- Suppose the research agent requests paid data beyond the classroom fixture.
- Are the purpose and supplier permitted? Is the fee within budget and the authorization still valid?
- Check who approved the purchase.
- AP2 illustrates signed payment authorization. QT itself has no payment access.

Source: AP2 v0.2 specification

Handle untrusted inputs



- Instructions found in filings or web pages cannot change the agent’s permissions.
- Enforce file and network restrictions through the host, not just the prompt.

Who is responsible?

Access
monitor systems and data

Oversight
define human checkpoints

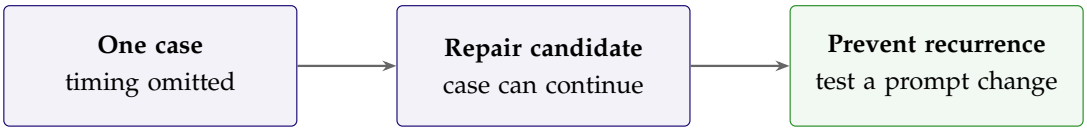
Records
trace actions and decisions

Limits
restrict and escalate

- Name the person responsible for reviewing exceptions and stopping the workflow when needed.
- FINRA offers a US broker-dealer example. The applicable obligations depend on the institution and its activities.

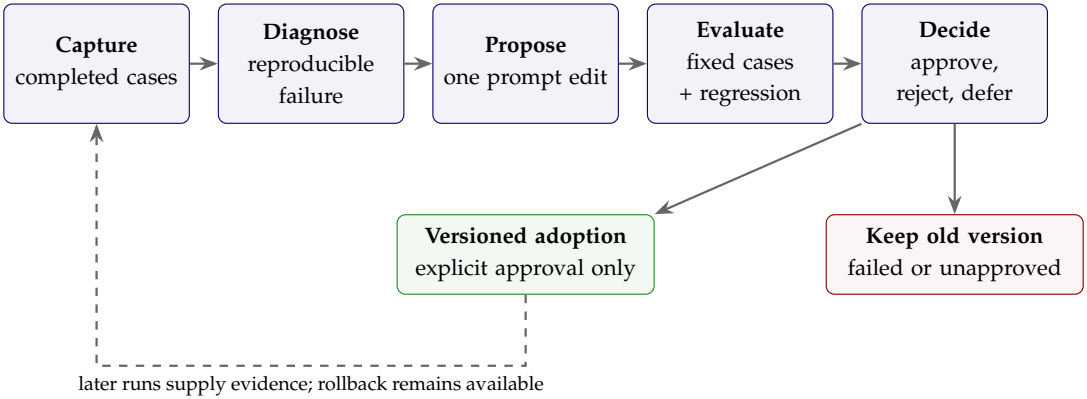
Source: FINRA 2026 Report, pp. 24–27

Prevent the same failure next time



- A repair fixes the current case. A workflow change aims to prevent the same omission in later cases.
- Base the proposed change on recorded failures.

Evaluate a change before adoption

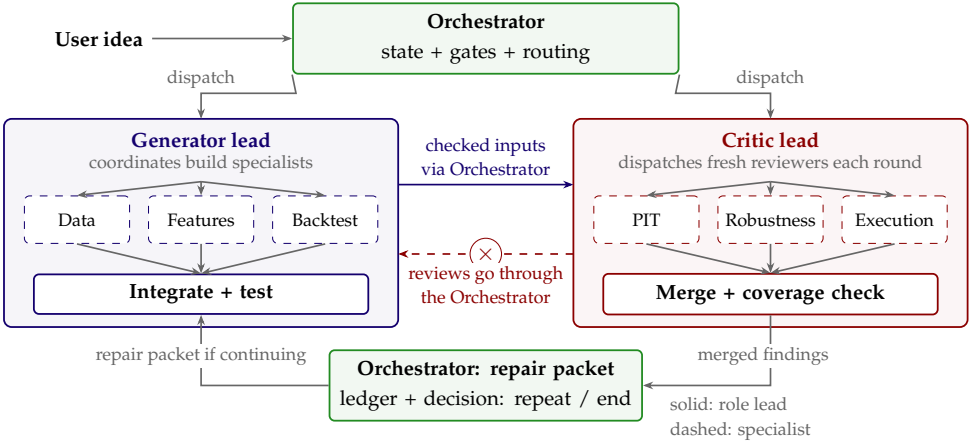


- Keep acceptance rules fixed and retain the previous version for rollback.
- QT currently versions only the Generator's role prompt.

What each loop changes

- The **tool loop** chooses the next action within a role.
- The **research loop** revises a candidate until the case ends.
- The **evolution loop** evaluates a reusable prompt change for adoption, rejection, or deferral.
- Changing a strategy parameter is a different task from improving the harness.

The complete architecture



- Research runs supply evidence; the owner reviews and approves version changes.

Table of contents

- **Design the quant system**
  - Controlled research
  - Recovery and governance
- **Build and improve the system**
  - **Run and verify QT**
  - Improve the system

Plugin files, commands, and installation

```
qt/
  kimi.plugin.json        # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/     # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/qt.py           # internal controller actions
  tests/                  # fixtures and regression tests
```

- Use `/qt:start` for research and `/qt:evo` for improvements.
- Install with `/plugins install <qt>`. After source edits, reinstall and run `/reload`.
- Keep installed code separate from case state and prompt versions in `.qt/`.

Assign leads and specialists

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- During a live run, inspect the host's dispatch and response records.
- Report files alone do not prove that separate specialists ran.

Register and inspect a hook

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "^Bash$",
      "command": "python3 ./hooks/audit_event.py",
      "timeout": 3
    }
  ]
}
```

- QT's optional Bash logger records event and call IDs without duplicating repeated deliveries.
- It does not assess candidates or provide a complete record of specialist execution.
- Kimi logs hook errors and timeouts, then continues. Required checks must remain in the controller.

Source: Kimi hook behavior

Follow one research case

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
  generator/*.json -> numerical/ -> critic/*.json
  state.json -> ledger and route -> final report
```

- Follow the `request_id`, specialist reports, numerical outputs, and routing decision.
- Label scripted rehearsals. Show host dispatch records when claiming a live team ran.

What must pass before the next step?

- All three Generator reports must match the current request.
- The candidate must run and pass the numerical checks.
- All three fresh Critic reviews must address the carried findings.
- The evidence must meet the acceptance rule or explain why the case stops.

Repair a rejected submission

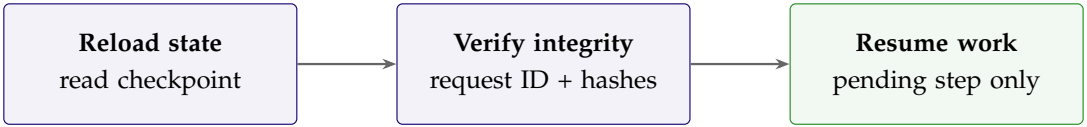
- If a specialist report is missing, keep the case at the current role.
- Reject a stale request ID without advancing state.
- Check matching reports, artifact hashes, and required conditions before proceeding.
- Repair a blocked submission in a disposable case. Keep sealed evidence intact.

Hands up

Check the saved evidence

- Inspect `state.json`, the route trace, and saved outputs to reconstruct the run.
- Repeat the numerical check with frozen inputs and compare results.
- Keep a separate record for each model run.
- Start a new case for a fresh evaluation. Reopening an existing case resumes or reuses its state.

Resume the pending step



- A completed case returns its saved result without repeating the work.
- Start a new case if the code, data snapshot, or role instructions change.

Table of contents

- **Design the quant system**
  - Controlled research
  - Recovery and governance
- **Build and improve the system**
  - Run and verify QT
  - **Improve the system**

Turn a failure into a test case

- Use `/qt:evo` (`evo-prepare`) to link the completed case to its failure evidence.
- Record the parent Generator prompt version.
- Define the required behavior: trade only after the signal is available.
- Choose development cases and reserve separate cases for the final evaluation.

Hands up

Propose one prompt change

```
  Describe the strategy and backtest results.
+ State when each signal becomes available.
+ State the earliest permitted execution time.
+ If timing evidence is missing, report the limitation.
```

Illustrative

- Check that the added instructions address the timing omission.
- Leave strategy parameters, risk limits, and acceptance rules unchanged.

Define the evaluation criteria

- Required fields must parse correctly.
- Trading must follow actual signal availability; a date field alone is insufficient.
- Cited sources must exist in the saved evidence and support the claims.
- Keep unauthorized actions blocked. Set these criteria before comparing prompts.

promptfoo: compare the outputs

Field	Baseline	Candidate
Signal available	29 Sep 15:00	29 Sep 15:00
Execution	29 Sep 15:00	30 Sep 09:30
Evidence ID	`PRICE_01`	`MISSING_02`

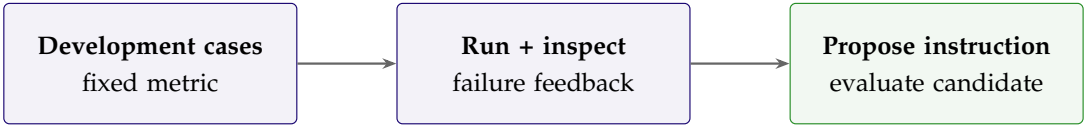
Illustrative

- In this example, trading is allowed from 30 Sep 09:30 (UTC+8). Only `PRICE_01` is retained.
- Run the companion checks, or load `promptfooconfig.json` in promptfoo, to compare both outputs.

Source: promptfoo configuration and assertions

Hands up

DSPy/GEPA: propose prompt changes



- GEPA uses a callable program, development cases, a metric, feedback, and a search budget to propose prompt changes.
- Test the candidates on held-out cases. A score does not authorize adoption.

Source: DSPy GEPA (external extension)

Check for regressions

- Run `python3 <qt>/scripts/check.py` to check controller rules.
- Compare old and new outputs against the timing and evidence checks.
- Reject changes to risk limits, strategy parameters, or controller code through this interface.
- Code tests cannot establish whether the prompt improved model behavior.

Decide from the evidence

Check	Baseline	Candidate
Execution timing	Fail	Pass
Evidence reference	Pass	Fail
Adoption decision	Reject	Reject

Illustrative

- The candidate fixes the timing but cites a missing source, `MISSING_02`.
- What must be repaired and checked before adopting it?

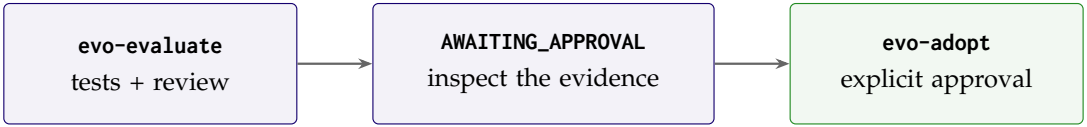
Hands up

Record the tested configuration

- QT stores immutable prompt versions and the active pointer in `.qt/`.
- MLflow can link prompt versions to evaluations and traces.
- Record the model, tools, policy, and data versions used in each run.
- A registry alias selects a version; approval needs its own record.

Source: MLflow Prompt Registry

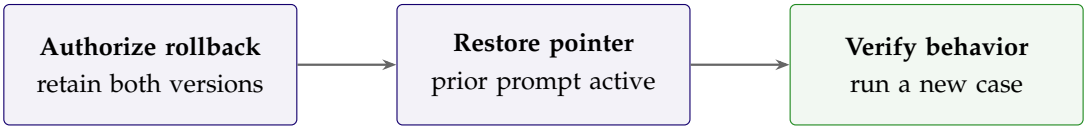
Approve a tested version



- QT checks code regressions and review fields. A person still needs to assess the comparison evidence.
- Would you adopt this version? Explain your decision using the saved results.

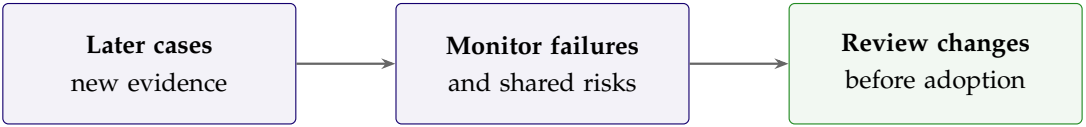
Hands up

Restore the previous version



- After approval, use `evo-rollback` to restore the previous pointer. Keep both versions and their evidence.
- Existing cases retain their original configuration; they do not switch versions mid-run.

Monitor later runs

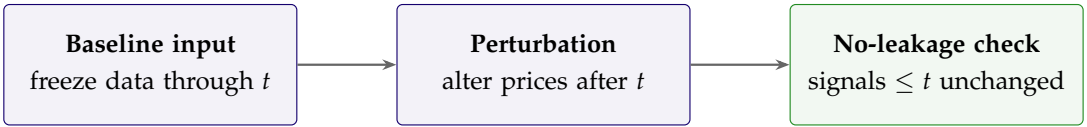


- Re-evaluate when models, tools, data feeds, or permissions change.
- Keep checking later runs for recurring failures. Prompt optimization does not guarantee reliability.

Replace synthetic data

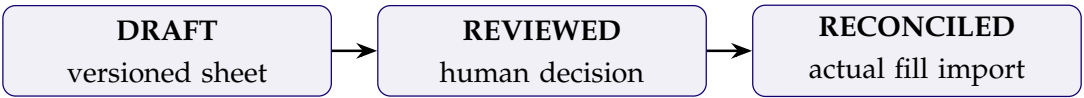
- Replace synthetic inputs with licensed, dated market and financial data.
- Reconstruct historical index membership, suspensions, and delistings.
- Apply lot sizes, price limits, T+1 rules, and fees for the relevant board and date.
- Add host isolation, name the responsible people, and test each production adapter.

Test the research pipeline



- Test future-price changes, restatements, higher costs, and performance in successive out-of-sample periods.
- Passing these checks does not establish that the strategy will earn excess returns.

Authorize and reconcile trades



- Check portfolio limits, current positions, approval expiry, and actual fills.
- Research acceptance does not authorize orders. A named person remains responsible for capital use.

Review the completed system

Evidence
trace and reproduce

Control
block or explain

Continuity
resume verified work

Improvement
evaluate and restore

- Choose one control and explain whose need it addresses.
- Show what has been tested, what the results support, and what still needs to be built.

Hands up